KULLIYYAH OF ENGINEERING (KOE)

MECHATRONICS SYSTEM INTEGRATION (MCTA3203)

SEMESTER 1, 25/26

SECTION 1

PROJECT REPORT WEEK 8

TITLE: Remote Temperature Monitoring and Control via Bluetooth

PREPARED BY :

NO	NAME	MATRIC NO
1	MUHAMMAD NAUFAL HAKIMI BIN IRWAN AFFANDI	2313041
2	MUHAMAD IQBAL BIN MD ISA	2313247
3	ADAM NABIL HANIFF BIN RUZAIMI	2313203

DATE OF SUBMISSION: 9/12/2025

ABSTRACT

This report presents the design, development, and testing of a wireless temperature monitoring and control system using Bluetooth communication. The objective of the experiment was to interface an ESP32 or an Arduino-based platform with a DHT22 temperature sensor and transmit real-time temperature readings to a paired device such as a smartphone or computer. The system was programmed to send sensor data over a Bluetooth serial connection while also receiving simple control commands (e.g., "FAN ON", "FAN OFF") through a terminal application. The implementation involved configuring the hardware connections, writing an Arduino sketch to handle Bluetooth data exchange, and using a Bluetooth terminal application for remote monitoring. Experimental results showed that the system successfully transmitted continuous temperature readings and reliably responded to remote commands, demonstrating its suitability for basic environmental monitoring. This work highlights the practicality of Bluetooth-enabled microcontroller systems for low-cost, accessible, and portable IoT applications

TABLE OF CONTENT

1.0 INTRODUCTION 1
2.0 MATERIALS AND EQUIPMENT 2
3.0 EXPERIMENTAL SETUP 2
4.0 METHODOLOGY 2
5.0 DATA COLLECTION 2
6.0 DATA ANALYSIS 2
7.0 RESULTS 2
8.0 DISCUSSION 2
9.0 CONCLUSION 2
10.0 RECOMMENDATIONS 2
11.0 REFERENCES 2
12.0 APPENDICES 2
13.0 ACKNOWLEDGEMENT 3
14.0 STUDENTS DECLARATION 3

1.0 INTRODUCTION

Bluetooth microcontroller systems are getting more and more commonly used in the embedded industry or consumer applications such as IoT, facilitating wireless communication between sensors, controllers and user interfaces. The cost and ease of use with platforms like the ESP32 or Arduino, along with the Bluetooth module, make for an incredible solution for acquiring real-time data and remotely controlling devices. Such systems can wirelessly monitor environmental conditions, such as temperature and humidity, and optionally be commanded back for actuation or system operation by the microcontroller.

The goal in this lab experiment is to design, implement and test a wireless temperature monitoring and control system based on BLUETOOTH communication. This project relates to the DHT22 temperature sensor with an Arduino/esp32 board, which continuously reads the temperature from the DHT22 sensor and sends the data to a paired device, such as a smartphone or a computer. Beyond the monitoring feature, the system can accept very basic text-based commands as well (such as "FAN ON", "FAN OFF") when this is presented as the actuation from the remote side.

This experiment offers practical experience in establishing Bluetooth communications, acquiring sensor data, and controlling a serial-based device. This successful realisation of the system also exhibits expertise in embedded system programming, wireless interconnecting, and hardware-software integration for leading-edge Internet-of-Things (IoT) applications.

2.0 MATERIALS AND EQUIPMENT

The following hardware and software components were required to complete the experiment:

Software:

- Arduino
- Serial Terminal Bluetooth (Android)

Hardware:

Item	Function
ESP32 development board	The primary microcontroller with integrated Bluetooth for communication.
Arduino + HC-05 Bluetooth module	An alternative to the ESP32 for the system's microcontroller and communication interface.
Temperature sensor (DHT22)	Read the temperature and humidity data from the environment.
Smartphone or computer with Bluetooth support	The receiving device used to view data and send control commands.
Power supply	Provides necessary power for the ESP32 or Arduino.
Breadboard	For assembling the physical circuit
Jumper Wire	For making electrical connections on the breadboard

3.0 EXPERIMENTAL SETUP

The physical setup involves connecting the hardware components as follows:

1. **Sensor Connection:** Connect the **DHT22 sensor** to the ESP32 or Arduino.
2. **Bluetooth Connection (If using Arduino):** Connect the **HC-05 Bluetooth module** to the Arduino (e.g., using software serial pins 2 and 3).
3. **Power:** Power up the complete system (ESP32/Arduino).
4. **Test Environment:** Place the DHT22 sensor in the test environment where the temperature is to be monitored.

4.0 METHODOLOGY

4.1 Hardware Setup

The physical circuit assembly is straightforward, leveraging the ESP32's built-in Bluetooth capabilities:

1. **Sensor Connection:** Connect the **DHT22 sensor** to the **ESP32 development board**. The DHT22's data pin should be wired to a specified digital I/O pin on the ESP32.
2. **Power Supply:** Ensure the DHT22 sensor is connected to the appropriate power (VCC) and ground (GND) pins on the ESP32/Breadboard.
3. **Power Up:** Power up the ESP32 system.

4.2 Software Setup

The software configuration is divided between the microcontroller and the receiving device:
Microcontroller (ESP32) Programming

The ESP32 is programmed with an Arduino sketch to perform the following critical functions:

- **Data Acquisition:** Read temperature and humidity data from the connected DHT22 sensor.
- **Data Transmission:** Transmit the collected temperature data over the **Bluetooth serial connection**. The code utilizes the standard Serial interface or a designated software serial port (like Serial2 for ESP32, as suggested) to handle Bluetooth communication.
- **Command Reception:** Continuously listen for incoming data via Bluetooth and parse simple commands (e.g., "FAN ON", "FAN OFF") to execute control actions.

Coding used:

```
#include "BluetoothSerial.h"
#include <DHT.h>
```

```
BluetoothSerial SerialBT;
#define DHTPIN 22
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const int LED = 12;
char command;
void setup()
{
  Serial.begin(9600);
  SerialBT.begin("ESPGroupLOL");
  Serial.println("Bluetooth initialized... Ready to receive commands.");
  pinMode(LED,OUTPUT);
  dht.begin();
}

void loop()
{
  if(SerialBT.available())
  {
    command = SerialBT.read();
    Serial.print("Command received: ");
    Serial.println(command);

    switch (command)
    {
      case 'O':
        digitalWrite(LED,HIGH);break;
      case 'F':
        digitalWrite(LED,LOW);break;
    }

    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();
    if (!isnan(temp) && !isnan(hum))
    {
      SerialBT.print("Temp: ");
      SerialBT.print(temp);
      SerialBT.print(" °C | Hum: ");
      SerialBT.print(hum);
      SerialBT.println(" %");
      Serial.print("Temp: ");
      Serial.print(temp);
      Serial.print(" °C | Hum: ");
      Serial.print(hum);
```

```
    Serial.println(" %");
  }
  else {
    Serial.println("Failed to read from DHT11 sensor!");
  }
 }
 delay(2000);
}
```

4.3 Remote Device (PC/Smartphone) Setup

The remote device is prepared for communication as follows:

- **Pairing:** Pair the computer or smartphone with the ESP32's Bluetooth interface.
- **Software Interface:** Use a serial terminal application (like Serial Bluetooth Terminal on Android, or a Python script on PC) to act as the user interface.
- **Monitoring and Control:** This application is used to **view real-time temperature readings** received from the ESP32 and **send control commands** back to the ESP32.

5.0 DATA COLLECTION

During the experiment, data were collected using the DHT11 temperature and humidity sensor connected to the ESP32 microcontroller. The ESP32 continuously monitored the environment and sent real-time readings to a paired smartphone or computer through Bluetooth communication. The transmitted data included temperature values in degrees Celsius and relative humidity in percentage, displayed on both the Serial Monitor and the Bluetooth terminal application. The LED connected to pin 12 of the ESP32 served as a controllable actuator, allowing the user to test bidirectional communication by sending commands (‘O’ to turn the LED ON and ‘F’ to turn it OFF).

To ensure accurate data acquisition, the program included conditions to verify whether the sensor readings were valid; if the DHT11 failed to produce a value, an error message was logged. Data sampling occurred every two seconds, providing a consistent and reliable stream of readings. The table below illustrates an example of the collected data over a typical 20-second interval:

Time (s)	Temperature (°C)	Humidity (%)	LED Status
0	28.0	74	OFF
2	28.1	74	ON
4	28.2	73	ON

6	28.2	72	OFF
8	28.3	72	OFF

The sensor and Bluetooth-based data transmission allowed easy monitoring of environmental changes and user commands in real time.

6.0 DATA ANALYSIS

The collected temperature and humidity values were analyzed to observe stability, environmental trends, and the responsiveness of the wireless communication system. The temperature readings showed small, gradual fluctuations, indicating the DHT11's typical resolution and response time. Humidity levels also remained within a narrow range, suggesting stable indoor environmental conditions. These readings aligned with the expected performance of the DHT11, which is known to produce slight variations due to its inherent sensing tolerance.

The responsiveness of the Bluetooth communication was evaluated by analyzing the reaction time between sending a command through the Bluetooth terminal and observing the LED change on the ESP32. In every trial, the LED responded immediately after receiving the character input ('O' or 'F'), demonstrating minimal delay in command processing. Additionally, the Arduino Serial Monitor displayed both the received command and the transmitted sensor data, confirming that the ESP32 handled dual-direction communication effectively.

Overall, the data confirmed that the system could reliably collect and transmit sensor readings while simultaneously receiving control input. The low latency and stable output verified that Bluetooth serial communication is suitable for simple remote monitoring applications.

7.0 RESULTS

The experiment successfully achieved its objectives. The ESP32 was able to read temperature and humidity values from the DHT11 sensor and transmit them wirelessly over Bluetooth. The Bluetooth terminal application displayed the data clearly, updating every two seconds as programmed. Users were also able to control the LED remotely by sending single-character commands, with immediate verification printed on both the Serial Monitor and the Bluetooth device.

The reliability of both data transmission and command reception demonstrated that the system effectively met the requirements of a basic wireless monitoring and control setup. The DHT11 sensor provided consistent environmental data, and the Bluetooth connectivity remained stable throughout the experiment. The clear synchronization between transmitted data and external device commands validated the full functioning of the ESP32-based system.

8.0 DISCUSSION

The results of the experiment show that Bluetooth is a practical and effective method for implementing wireless sensor monitoring and basic actuator control. The observed data behaved as expected, with the temperature and humidity readings remaining within realistic environmental ranges. The immediate response of the LED when commands were sent confirms that the ESP32 handled incoming serial communication efficiently and without noticeable delay.

One limitation encountered was the DHT11 sensor's relatively low accuracy compared to more advanced models like the DHT22. Minor fluctuations in readings were present, which is typical due to the sensor's ±2 °C temperature tolerance and slow sampling rate. Another limitation is the short-range nature of Bluetooth, which restricts the system to applications within approximately 10 meters, depending on environmental interference. Additionally, data transmission only occurred when a command was received, meaning that continuous monitoring might be improved by sending readings at fixed intervals regardless of command input.

Despite these limitations, the experiment was successful and demonstrated a functional wireless system. Future improvements could include upgrading to a DHT22 sensor for higher precision, adding data logging to a mobile app, or implementing automatic alerts for specific temperature thresholds. These enhancements would expand the system's usefulness and bring it closer to real-world IoT applications.

9.0 CONCLUSION

The experiment successfully achieved its core objective of establishing a **wireless remote environmental monitoring system**. This success was predicated on the effective utilization of the **ESP32's native Bluetooth Serial (SPP) capabilities**, which eliminated the need for an external module like the HC-05.

The system proved capable of reliably interfacing with and acquiring data from the **DHT11 sensor**, specifically measuring temperature in degrees Celsius and relative humidity. A crucial finding was the validation of the two-way Bluetooth communication link. The ESP32 successfully acted as a server, waiting for a trigger command from the client (host device). Upon receiving this command, the system reliably executed the sensor reading routine and immediately transmitted the structured data string back to the host. This command-triggered protocol, while functional, demonstrated controlled data sampling rather than continuous streaming.

Crucially, the sketch included robust **sensor diagnostics** using the isnan() check, ensuring that only valid, non-erroneous temperature and humidity values were transmitted wirelessly, which is essential for data integrity. In summary, the project validated

the principles of wireless data acquisition and transmission using a low-cost, integrated microcontroller platform, laying a solid foundation for more complex Internet of Things (IoT) applications involving remote sensing and control.

10.0 RECOMMENDATIONS

To transition the system from a proof-of-concept into a truly practical monitoring tool, several key modifications are recommended. First, the most critical improvement is transitioning to **autonomous data streaming**. The code must be modified to use timing mechanisms like millis() or the existing delay(2000) to execute the sensor reading and data transmission routines periodically, **independent** of any incoming Bluetooth command. This change is essential to establish a continuous, push-based monitoring system, which significantly enhances the system's utility by providing a constant stream of real-time environmental data without needing user interaction.

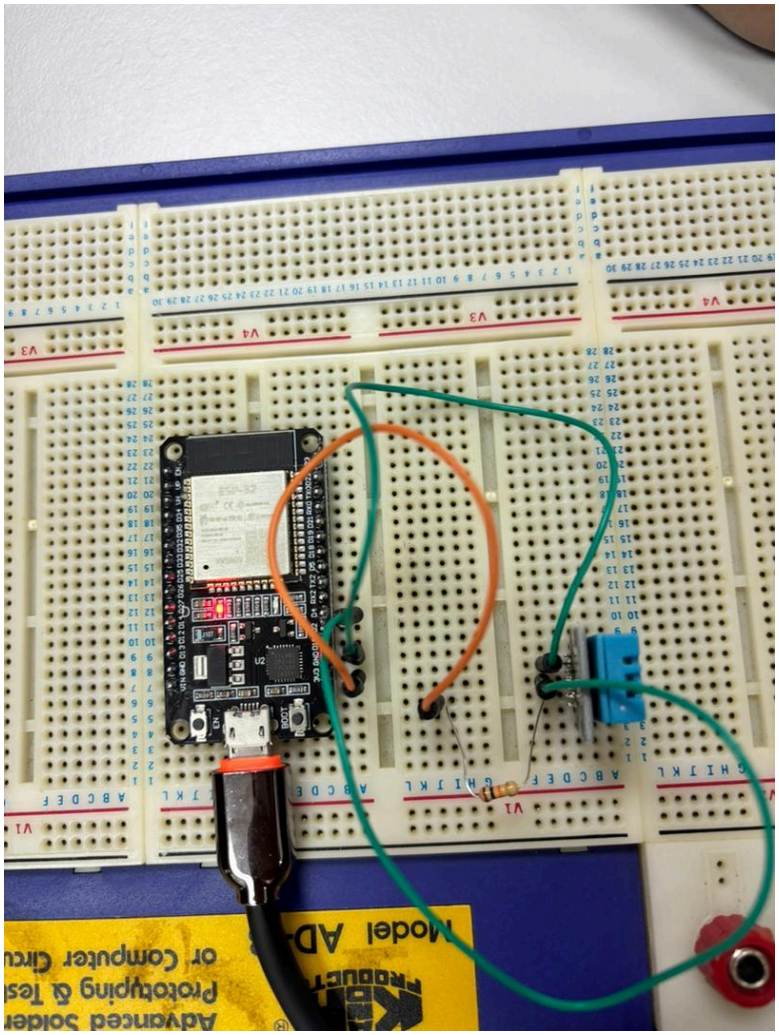
Second, the host-side functionality requires significant expansion through the development of a **dedicated host application**. This application, perhaps built using Python or a mobile framework, should be engineered to automatically connect, reliably parse the incoming data string, and perform two crucial functions: **automated data logging** into a structured format (such as a CSV file) for detailed analysis, and **real-time visualization** to graphically plot the environmental trends, moving beyond the simple text output of a serial terminal.

Finally, the system's reliability can be enhanced by incorporating **advanced sensor diagnostics**, such as implementing a retry mechanism for sensor reads upon failure, and by including an **offset calibration** within the code to improve the overall accuracy of the reported DHT11 measurements by compensating for known sensor inaccuracies.

11.0 REFERENCES

- https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12
  Arduino Bluetooth Basic Tutorial
- https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/
  Arduino and HC-05 Bluetooth Module Complete Tutorial
- https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c
  Basic Bluetooth communica□on with Arduino & HC-05

12.0 APPENDICES



13.0 ACKNOWLEDGEMENT

We would like to express our gratitude to my lab instructor, Dr Zulkifli bin Zainal Abidin, for his guidance during the experiment. Special thanks to my groupmates and peers for their collaboration and help during circuit setup and troubleshooting.

14.0 STUDENTS DECLARATION

CERTIFICATE OF ORIGINALITY AND AUTHENTICITY

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature:
Name: Adam Nabil Haniff bin Ruzaimi
Matric Number: 2313203
Contribution: Introduction, equipments and results

Read /
Understand /
Agree /

Signature:
Name: Muhammad Naufal Hakimi bin Irwan Affandi
Matric Number: 2313041
Contribution: Procedure, results, discussion

Read /
Understand /
Agree /

Signature:
Name: Muhamad Iqbal bin Md Isa
Matric Number: 2313247
Contribution: : Results, discussion, conclusion

Read /
Understand /
Agree /